

Programação Assíncrona: Callbacks, Promises e Async/Await

Objetivo da Aula

Conhecer os ferramentais oferecidos pelo JavaScript para programação no modelo assíncrono, bem como utilizá-los em contextos práticos no NodeJS.

Apresentação

Ao trabalhar com operações de I/O, não pode ocorrer o bloqueio de execução no fluxo principal, pois todo o sistema ficaria ocioso, e as operações assíncronas são uma solução elegante para tratar o paralelismo.

Na programação JavaScript, o retorno de um processo assíncrono será um elemento do tipo promise, que representa a esperança de um valor, em algum instante não determinado, quando é finalizada a execução do thread.

Veremos, então, como implementar funções assíncronas e como as palavras “async” e “await” podem ser utilizadas na construção de processos assíncronos, com base em uma sintaxe bastante simples.

Também iremos verificar o papel das funções de callback, na programação assíncrona, com o tratamento dos dados recebidos, e conheceremos as diversas palavras reservadas voltadas para o modelo assíncrono.

1. Funções de Callback

Um processo **assíncrono** executa em um **thread** separado e deve ter seu resultado tratado após o término de sua execução, mas, como são processos em paralelo, devemos ter um meio de comunicar a saída da operação com o fluxo que a iniciou. Nesse contexto, temos as funções de callback, que são utilizadas para receber e tratar algum resultado fornecido por um processo assíncrono.

Em um fluxo com processamento síncrono, podemos utilizar o operador **then** na chamada de uma função **assíncrona**. Esse operador captura o retorno e permite especificar uma função de tratamento, ou **callback**, mas não bloqueia a execução do fluxo principal, viabilizando que o tratamento da informação ocorra apenas quando ela for recebida.

```
const fsPromises = require('fs').promises;
fsPromises.readFile("./Meu_Arquivo.txt")
.then((result) => {
  console.log("Lido: " + result);
})
.catch(function (error) {
  console.log(error);
})
```

No exemplo é feita a leitura assíncrona de um arquivo, utilizando **readFile**, a partir da biblioteca **fs**, na sua forma de **promises**, e o conteúdo lido é impresso no console através de uma **callback** no operador **then**, enquanto um erro de I/O levaria à impressão do erro pela callback do operador **catch**.

As funções de **callback** são a base para o tratamento assíncrono de dados, e o que temos, na prática, é apenas uma função que fica preparada para receber os dados de um processo assíncrono. Temos utilizado callbacks em diversos de nossos exemplos, e isso é uma característica comum ao lidar com as operações de I/O, como no tratamento de arquivos e chamadas HTTP.

Outro exemplo de leitura de arquivo assíncrona é apresentado a seguir, com a passagem da **callback como parâmetro** da função de leitura. Nesse segundo modelo de implementação, não utilizamos o operador **then**.

```
const $fs = require('fs');
$fs.readFile('/Users/denis/teste.txt', 'utf8', (err, data) => {
  if(err) {
    console.error(err);
  }
  return;
```

```
}  
console.log(data);  
});
```

Na leitura assíncrona do arquivo, você terá um retorno, que pode ser um erro ou o texto do arquivo, sendo recebidos respectivamente nos parâmetros **err** e **data**. Caso ocorra o erro, ele será exibido no console, encerrando a execução da callback com return, e, no caso contrário, o texto será exibido.

O padrão **arrow function** tem uma escrita simplificada, mas podemos usar o formato tradicional de funções, como na listagem seguinte.

```
let router = express.Router();  
router.use(function timeLog(req, res, next) {  
  console.log('Time: ', Date.now());  
  next();  
});  
router.get('/', function(req, res) {  
  res.send('Birds home page');  
});
```

Foram definidas duas funções de callback, em que uma delas é utilizada como **middleware** do roteador e a outra responde à rota **raiz** via método **GET**. Note que a primeira recebe o nome **timeLog**, mas é apenas informativo, pois a função tem escopo local e não pode ser utilizada em outros pontos do código; ainda, mesmo retirando o nome, continuaria funcionando.

A primeira callback será executada sempre que houver uma solicitação para algum endereço, a partir do caminho de base do roteador, independentemente da rota utilizada. Por exemplo, se for utilizada por um servidor express, tendo como base a rota **/birds**, uma chamada para **http://localhost:3000/birds/aaa** retornaria um erro padrão **404**, mas teria logado o momento de acesso à rota.

Por outro lado, a segunda callback oferece a resposta na **raiz**, e, utilizando a mesma rota de base **/birds**, uma chamada para **http://localhost:3000/birds** iria acionar o middleware,

imprimindo o momento de acesso, e, logo após, passaria pelo tratamento da segunda callback, com o retorno de informação via **send**.

É importante observar como as callbacks nos permitem oferecer o tratamento adequado para os dados, sem a preocupação com o momento em que serão utilizadas. Em um ambiente síncrono, teríamos que esperar indefinidamente por cada evento, bloqueando o fluxo principal de execução, o que tornaria inviável a construção de um servidor.

As callbacks também são utilizadas para responder ao sistema de eventos do NodeJS, como no exemplo seguinte.

```
const libEmitter = require('events')
const myEmitter = new libEmitter.EventEmitter();
let m = 0;
myEmitter.on('somar', () => {console.log(++m);});
myEmitter.on('error', (err) => {
  console.error('Opa! Erro!');
});
myEmitter.emit('somar');
myEmitter.emit('somar');
myEmitter.emit('error', new Error('Erro!'));
```

No exemplo, temos a definição de duas callbacks, para os eventos **somar** e **error**, em que o primeiro evento incrementa um contador e imprime o valor final no console, enquanto o segundo recebe uma **exceção** e imprime uma mensagem indicando a ocorrência do erro. No caso específico do evento de erro, se ele não fosse capturado em **err**, ocorreria a interrupção da execução do NodeJS.

Executando o exemplo, teríamos a emissão de **dois** eventos do tipo **somar** e **um** evento do tipo **error**, com a passagem de uma **exceção**. Isso irá causar a impressão dos valores 1 e 2, além de uma mensagem indicando a ocorrência de uma exceção ao final.

2. Funções Assíncronas com Async e Await

A forma mais simples de definir uma função assíncrona é através da palavra reservada **async**, a qual tratará o resultado da função como **promise**, que é uma esperança de retorno, e não o retorno em si. Por executar em thread separado, o corpo da função **async** aceita o operador **await**, que espera o retorno de outro processo assíncrono, com bloqueio de execução.

Você pode observar um trecho de código assíncrono na listagem seguinte.

```
const somar = async (a,b) => a + b;
const imprimir_soma = async(a,b) => {
  let valor = await somar(a,b);
  console.log(valor);
  return "Processo Concluido";
}
imprimir_soma(1,2).then((retorno) => console.log(retorno));
```

No exemplo, temos uma primeira função assíncrona, com o nome **somar**, que retorna a soma dos valores de seus parâmetros via **promise**. A segunda função, também assíncrona, com o nome **imprimir_soma**, espera o retorno da primeira com **await**, ocorrendo o bloqueio de imprimir_soma até o retorno de soma.

Ao receber o valor com **await**, a camada promise é removida, utilizando-se do valor de retorno diretamente, sem a necessidade de uma callback. No caso da ocorrência de uma exceção, ela precisaria ser tratada em uma estrutura do tipo **try** e **catch**, equivalendo ao uso do operador catch em conjunto com then.

Na parte final, temos a chamada para imprimir_soma, com os valores 1 e 2, na parte síncrona, em que o operador **then** irá capturar o retorno e direcionar para a função de callback, que imprime o valor recebido. Sempre devemos lembrar que o uso de then não bloqueia o fluxo e que, se tivéssemos a impressão de algum conteúdo na linha seguinte àquela que invoca a função assíncrona, poderíamos ter a impressão desse conteúdo antes das demais impressões.

As funções que aceitam o uso de **then** podem ser invocadas com **await**, já que ambos os modelos trabalham com retorno do tipo **promise**.

```
const fsPromises = require('fs').promises;
const func_exemplo = async () {
  console.log(await fsPromises.readFile("./Meu_Arquivo.txt"));
}
```

Observando o código, temos a definição da função assíncrona, utilizando **async**, e, no corpo da função, ocorre a impressão do conteúdo de um arquivo, que é lido de forma assíncrona por **readFile**. Para que a impressão do conteúdo aguarde o término da leitura, foi utilizado o operador **await** na chamada.

Agora, vamos observar como ocorre o tratamento de exceções, para essa abordagem, na construção de funções assíncronas.

```
const $fs = require('fs/promises');
async function teste_leitura() {
  try {
    const data = await $fs.readFile(
      '/Users/denis/teste.txt', {encoding: 'utf8'});
    console.log(data);
  } catch (err) { console.log(err); }
}
teste_leitura();
```

Note que o fluxo de `teste_leitura` é bloqueado com **await**, esperando o retorno de **readFile**, exigindo, para o tratamento de exceções, o uso do bloco protegido por **try** e a captura do erro via **catch**. No caso da utilização em trecho síncrono, iríamos precisar dos operadores **then** e **catch**, como apresentado a seguir.

```
const $fs = require('fs/promises');
$fs.readFile('/Users/denis/teste.txt', {encoding: 'utf8'})
  .then((data)=>console.log(data))
  .catch((err)=>console.log(err));
```

O uso de **async** é um grande facilitador em respostas HTTP que dependem de operações de I/O, como acesso ao banco de dados, por permitir que threads sejam sincronizados com o uso de **await**.

```
const express = require('express');
const db = require('./models/index.js');
const router = express.Router();
router.get('/', async(req,res)=>{
  let dados = await db.Nota.findAll();
  res.json(dados);
})
```

No exemplo, temos um roteador respondendo na rota raiz, pelo método GET do HTTP, através de uma **callback assíncrona**. Como o envio da informação necessita de acesso ao banco de dados, aqui utilizando uma entidade Sequelize, o uso de **await**, para aguardar o recebimento dos dados, estabelece a sincronia entre os processos para que a resposta tenha os dados necessários.

3. Funções Assíncronas com Promises

Mesmo uma função comum pode ter comportamento assíncrono, e, para isso, basta utilizar um retorno do tipo **promise**. Um objeto desse tipo pode oferecer uma resposta válida, através do método **resolve**, ou emitir uma exceção, por meio do método **reject**.

Todo retorno definido em **resolve** será capturado na cláusula **then**, enquanto um erro gerado por **reject** será interceptado pela cláusula **catch**. Caso a função seja invocada a partir de uma função marcada como **async**, o retorno é obtido utilizando **await**, e os erros devem ser tratados em estruturas com **try** e **catch**.

Observe o exemplo que é apresentado a seguir.

```
const obterTodos = async() => {
  if(Math.random()<0.5)
    return [4,2,6,4,2,1,4,6,8,9];
}
```

```
throw "Erro ao recuperar";
}
const obterPromise = () => {
  const promise = new Promise((resolve, reject)=>{
    obterTodos()
      .then((dados)=>resolve(dados))
      .catch((err)=>reject(new Error(err)))
  })
  return promise;
}
obterPromise()
  .then((dados)=>console.log(dados))
  .catch((err)=>console.log(err))
```

Diversos elementos assíncronos estão presentes no exemplo, a começar pela função **obterTodos**, marcada como **async**. Na função, temos a geração de um número aleatório entre 0 e 1, oferecendo **50%** de chances para o retorno de um **vetor** ou, no caso contrário, a geração de um erro com **throw**.

A função **obterPromise** apresenta um retorno do tipo **promise**, em que temos, no corpo da resposta, a chamada para **obterTodos**, com uso de **then**, em que o vetor fornecido é enviado para o chamador via **resolve**, além de uma cláusula **catch**, que usa **reject** para propagar o erro ocorrido em um objeto **Error**.

Ao final, testamos nossa função baseada em promise, através das cláusulas **then** e **catch**, com a impressão do resultado no console. Parte das execuções irá imprimir o **vetor**, enquanto outra irá indicar **erro ao recuperar**.

Um retorno promise também pode ser definido diretamente, através de uma sintaxe simplificada, como no exemplo seguinte.

```
const somar = (a,b) => Promise.resolve(a+b)
const dobrar = (x) => Promise.resolve(x*2)
const dividir = (a,b) =>
```

```
(b==0)? Promise.reject(new Error("Div by zero")):
Promise.resolve(a/b);
```

As funções **somar** e **dobrar** retornam diretamente os resultados via **resolve**, enquanto a função **dividir**, com base em uma condição, pode utilizar **reject** para emitir um erro ou retornar a divisão através de **resolve**. Note como a sintaxe aqui utilizada é bem mais simples que o uso de uma instância de promise.

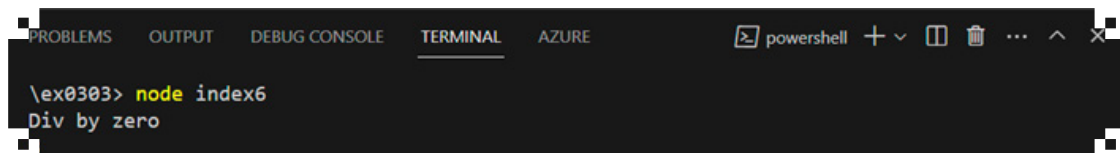
Vamos observar um exemplo de utilização das funções anteriores.

```
somar(4,5)
  .then((x)=>dobrar(x))
  .then((y)=>dividir(y,6))
  .then((z)=>console.log(z))
dividir(2,0)
  .then((x)=>console.log(x))
  .catch((err)=>console.log(err.message))
```

A primeira operação utiliza o encadeamento para transferir o retorno de uma função para a execução seguinte. Com isso, os valores 4 e 5 são somados, o resultado da soma é dobrado, depois dividido por 6, e, finalmente, o valor final do conjunto de operações, no caso 3, é impresso no console.

Já na segunda operação, forçamos um erro, que é capturado na cláusula **catch**, sendo impresso no console. Por causa da prioridade definida no ciclo de vida do **Event Loop**, o erro será impresso primeiro, como pode ser observado na figura apresentada a seguir.

Figura 1: Execução do exemplo com Promises



Fonte: Elaboração própria.

Tome cuidado ao utilizar **then**, pois não ocorrerá o bloqueio de execução no fluxo principal. Por exemplo, se efetuarmos a leitura assíncrona de um arquivo grande e, na linha seguinte ao then, tentarmos imprimir seu conteúdo, certamente a leitura não estará completa, já que ocorre em paralelo.

Podemos transformar o retorno de qualquer função em promise, utilizando a função **promisify** da biblioteca **util**. Na prática, essa função apenas encapsula o retorno da função original em um objeto do tipo promise.

O exemplo seguinte demonstra como transformar a escrita de arquivo para o formato de **promise**, viabilizando o uso das cláusulas **then** e **catch**.

```
const {promisify} = require('util');
const {writeFile} = require('fs');
const writeFilePromises = promisify(writeFile);
writeFilePromises('arquivo.txt', 'conteúdo do arquivo')
  .then(() => console.log('arquivo criado com sucesso!'))
  .catch(err => console.log(err));
```

A única restrição para transformar uma função é que ela deve utilizar o retorno na forma de **callback**, tendo os parâmetros erro e retorno. Na listagem seguinte, pode ser observado um exemplo desse estilo de programação.

```
const {promisify} = require('util')
const somar = (a,b,callback) => {
  const x = a + b;
  callback(null,x);
}
// Utilização padrão
somar(1,2,(err,w)=>console.log(w));
// Utilização via then
```

```
const somarPromise = promisify(somar);  
somarPromise(1,2).then((x)=>console.log(x));
```

Uma função com comportamento baseado em callback não retorna o valor, mas invoca a callback fornecida com a passagem do valor. Para que se adeque ao modelo utilizado por **promisify**, deve solicitar, como parâmetros, um campo para o erro e outro para o retorno.

Considerações Finais da Aula

Como pudemos ver, de forma prática, as funções de callback são necessárias para o tratamento de dados retornados por processos assíncronos. Elas ficam disponíveis, iniciando sua execução apenas quando ocorre o retorno da função assíncrona e o consequente término de seu thread, simplificando o controle da execução em paralelo.

Vimos que o retorno de uma função assíncrona é um elemento promise e que as palavras `async` e `await` permitem a implementação de forma mais simples e direta, já que o retorno é tratado automaticamente como promise.

Conhecemos as várias formas de utilização de processos assíncronos e a preocupação do JavaScript com esse modelo de desenvolvimento, fornecendo diversos operadores específicos. Pudemos compreender, também, a diferença do comportamento bloqueante, no uso de `await`, para as chamadas sem bloqueio, baseadas na cláusula `then`.

Material Complementar



JavaScript Assíncrono

2023, Mozilla Developer.

O site faz parte do portal do Mozilla Developer, trazendo informações acerca da programação assíncrona em JavaScript e da sua utilização na construção de APIs para o ambiente web. Link para acesso: <https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Asynchronous> (acesso em: 13 set. 2023).

Referências

BODRIN, M. **Processamento paralelo e distribuído**. Porto Alegre: SAGAH, 2021. Disponível em: <https://abre.ai/gKcQ>. Acesso em: 13 set. 2023.

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKcR>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKcW>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKcX>. Acesso em: 12 set. 2023.